🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 12: Fuzzing an Application with zzuf
```

Scenario

Application Fuzzing with zzuf is a transparent application input fuzzer that mutates file streams, standard input (`stdin`), and command-line data to discover memory corruption vulnerabilities, unexpected application crashes, and unhandled exceptions.

Key concepts in zzuf automated fuzzing:
1. **Reproducible Bit Mutation**: `zzuf` intercepts binary file operations and alters a precise ratio of bits determined by the `-r` flag (e.g., `-r 0.004` for 0.4% bit modification, `-r 0.05` for 5%). Fuzzing patterns are fully deterministic based on random seed values (`-s`).
2. **Exclusion & File Interception**: Using `-c` restricts fuzzing specifically to command-line argument files, while `-E` excludes shared system libraries or configuration paths (`/etc/`, `/usr/share/`) from input corruption.
3. **Crash Triage & Valgrind Analysis**: When `zzuf` triggers a Segmentation Fault (**SIGSEGV / Signal 11 / Signal 9**), exact seed parameters (`-s <seed>`) are extracted to generate a reproducible malformed test case file (`crash.exe`) for root-cause analysis under memory debuggers (**valgrind -q**).

In this exercise, we authenticate to the **Ubuntu 20.04 Fuzzing** VM (**student** / **studentpassword**), disable ASLR (`kernel.randomize_va_space=0`), examine `zzuf` manual options, execute reproducible bit mutations on `/dev/zero`, generate batch malformed file sets, fuzz `objdump` against Windows PE binary **win9x.exe**, isolate crashing seed `s=16915`, generate **crash.exe**, and perform memory crash triage using `valgrind`.

Objective

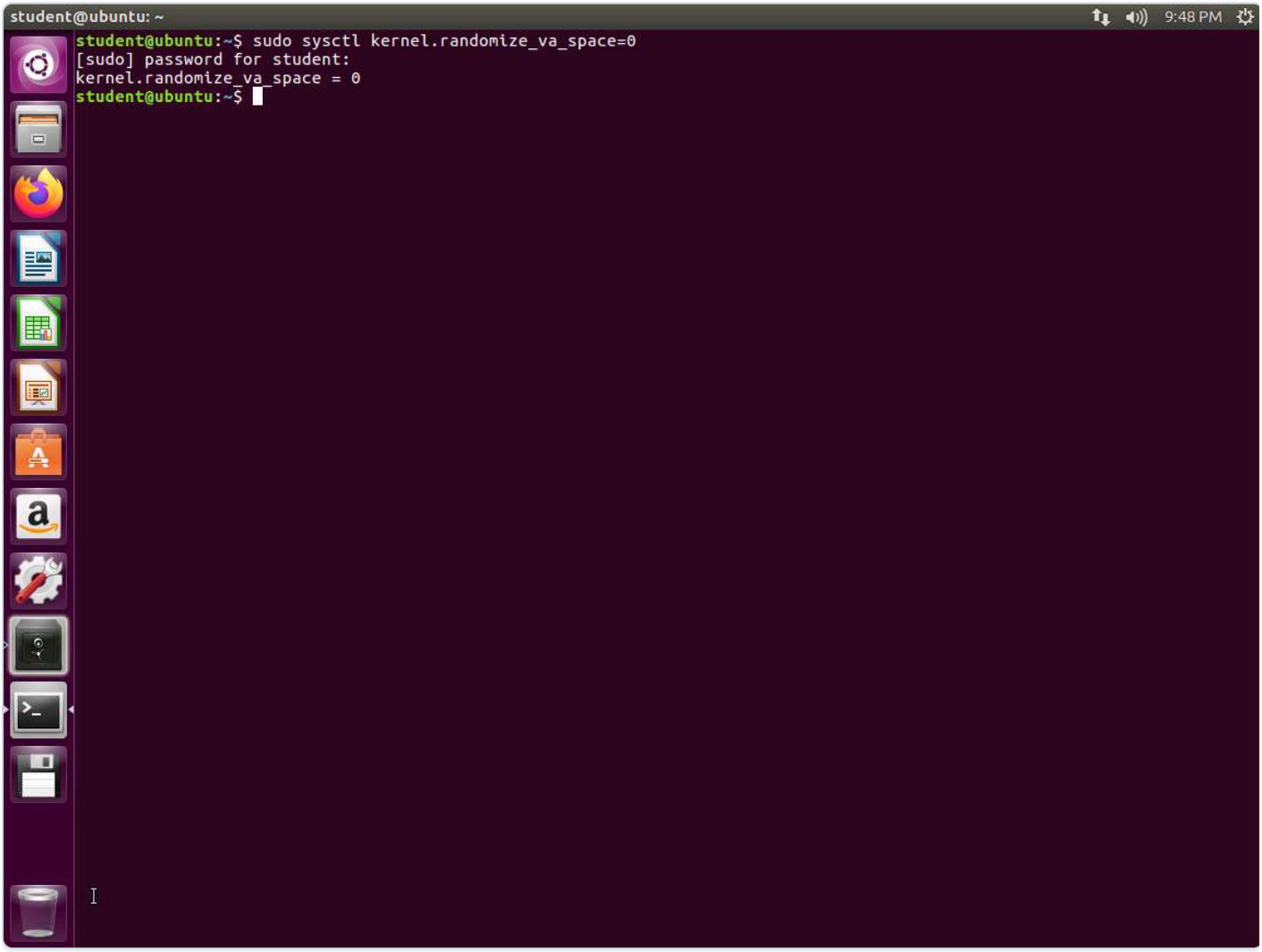
The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu 20.04 target VM (**student** / **studentpassword**)
- Disable Address Space Layout Randomization: **sudo sysctl kernel.randomize_va_space=0**
- Inspect manual documentation: **man zzuf**
- Perform reproducible stdout fuzzing: **zzuf cat /dev/zero | hd -vn 32**
- Adjust bit-mutation ratio (`-r 0.05`) and random seed values (`-s 10`)
- Construct malformed input files using `dd` and output redirection
- Filter system paths with exclusion flags: **zzuf -d -E /etc/ -E /usr/share/ file /bin/ls**
- Restrict fuzzing to command-line files: **zzuf -d -c file /bin/ls**
- Execute batch file generation loops producing 2,000 malformed input variants
- Execute high-volume batch fuzzing against `objdump -x win9x.exe` using `zzuf -s 0:1000000 -c -C 0 -q -T 3`
- Capture crashing seed value (`s=16915`) and signal number (**9** / **11**)
- Replicate crash payload into **crash.exe** and analyze under **valgrind -q**

Task 1: Disable ASLR, Execute Stream & File Fuzzing with zzuf, and Perform Crash Analysis

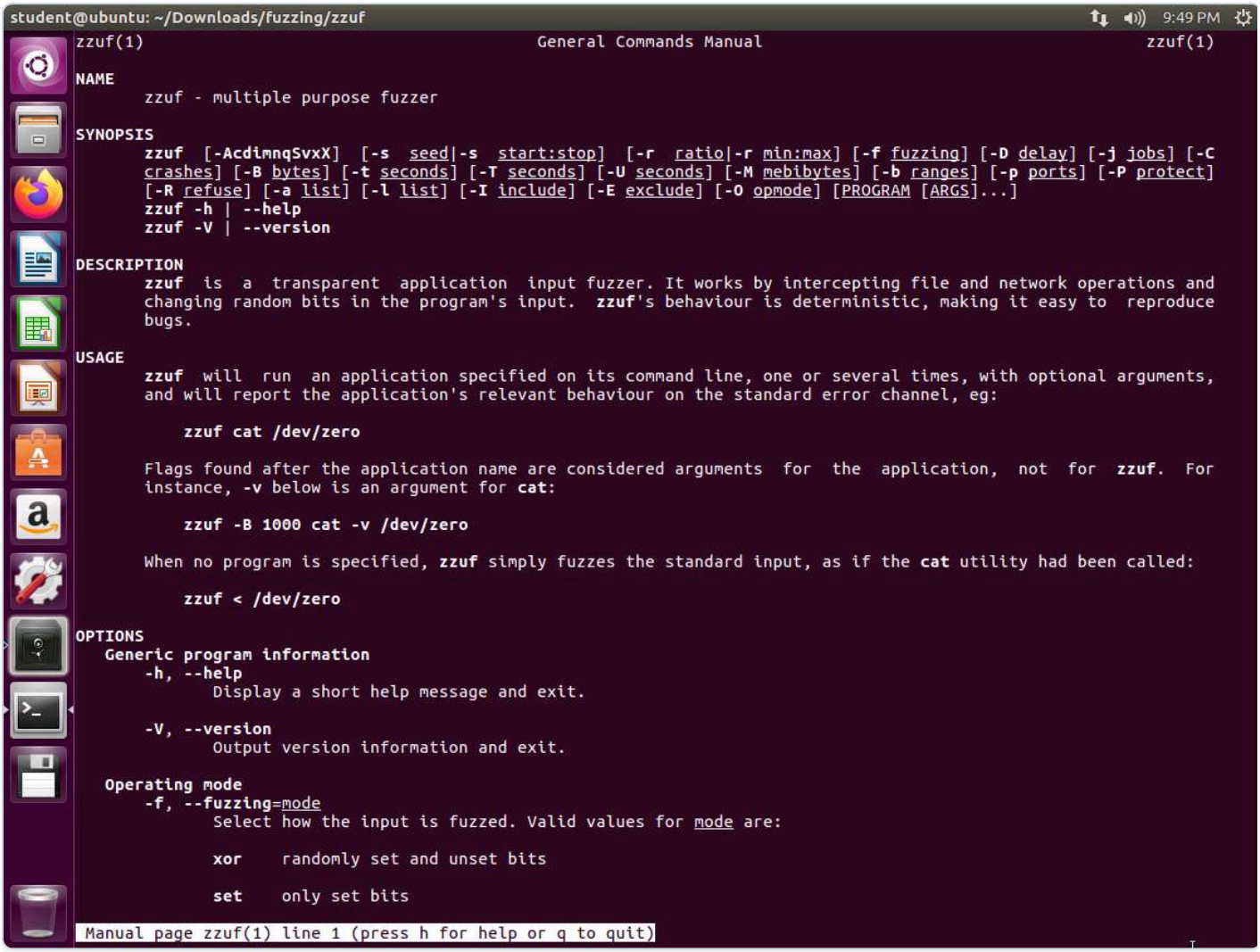
In this task, we disable ASLR on Ubuntu 20.04, experiment with bit ratios and seeds, generate malformed batch files, fuzz `objdump -x win9x.exe`, isolate crashing seed `16915`, and analyze crash memory dumps with `valgrind`.

Lab Tasks

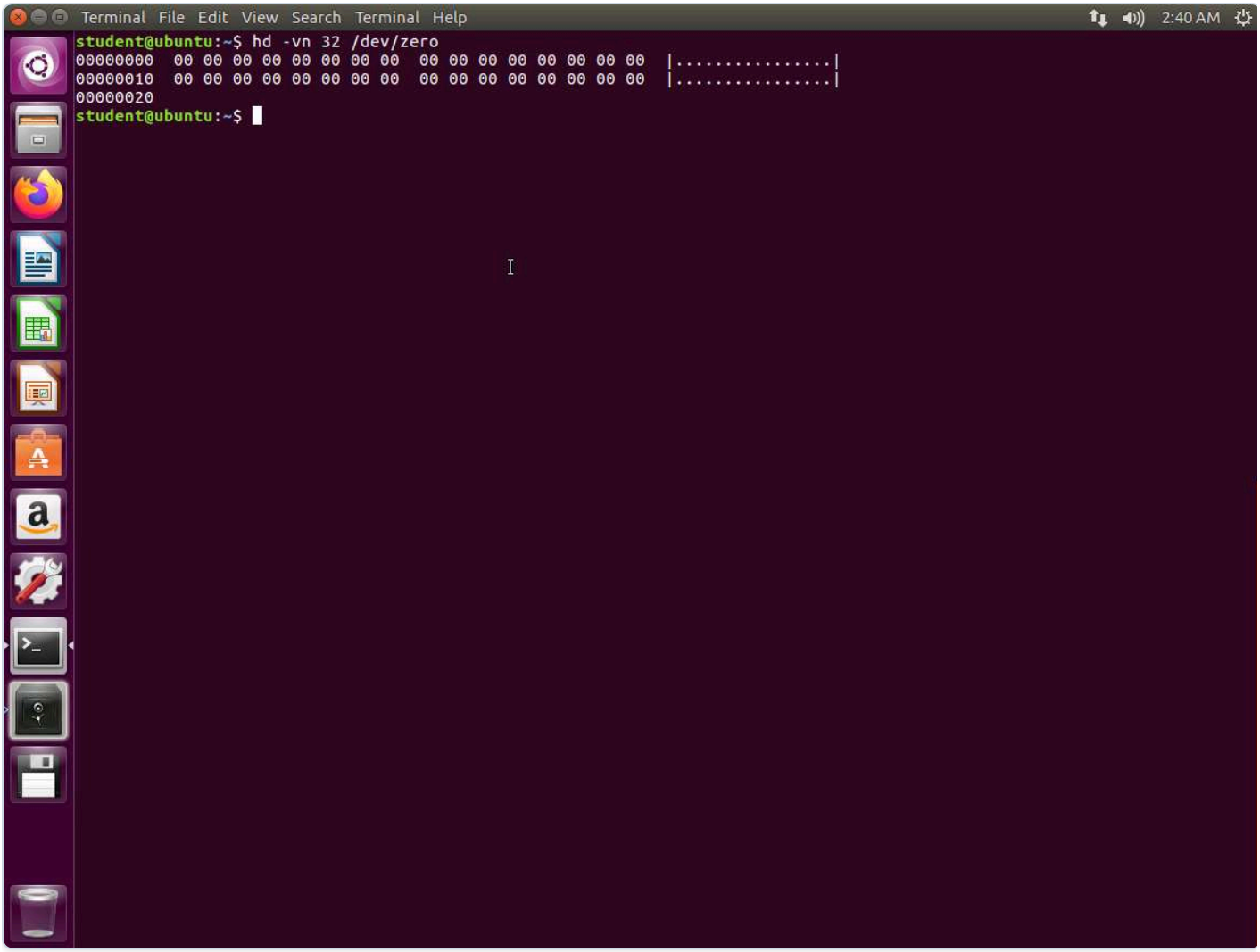
1. Log into **Ub20 Fuzzing** VM using credentials (**student** / **studentpassword**). Open terminal and disable ASLR:
   sudo sysctl kernel.randomize_va_space=0



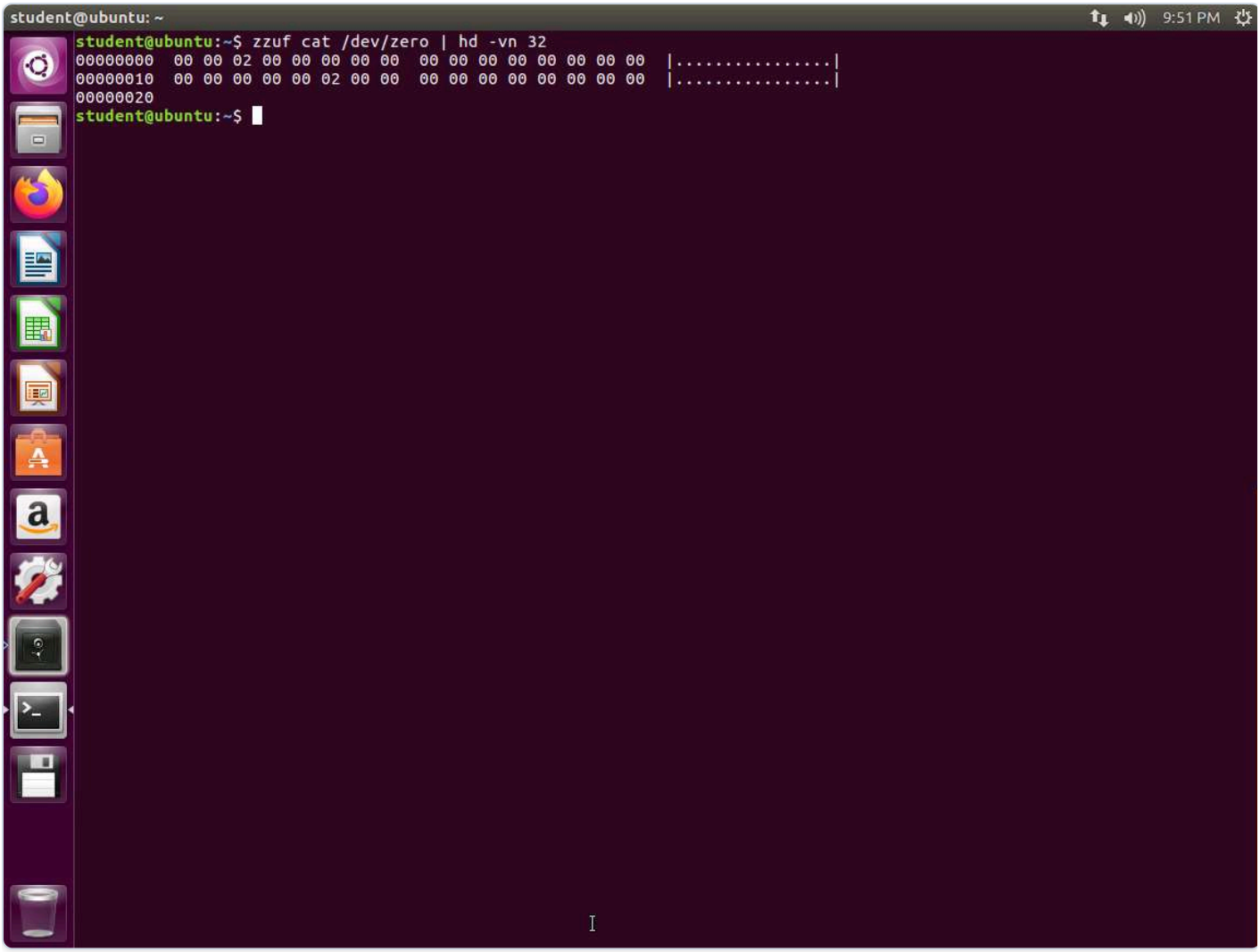
1. Inspect `zzuf` command manual page:
   man zzuf



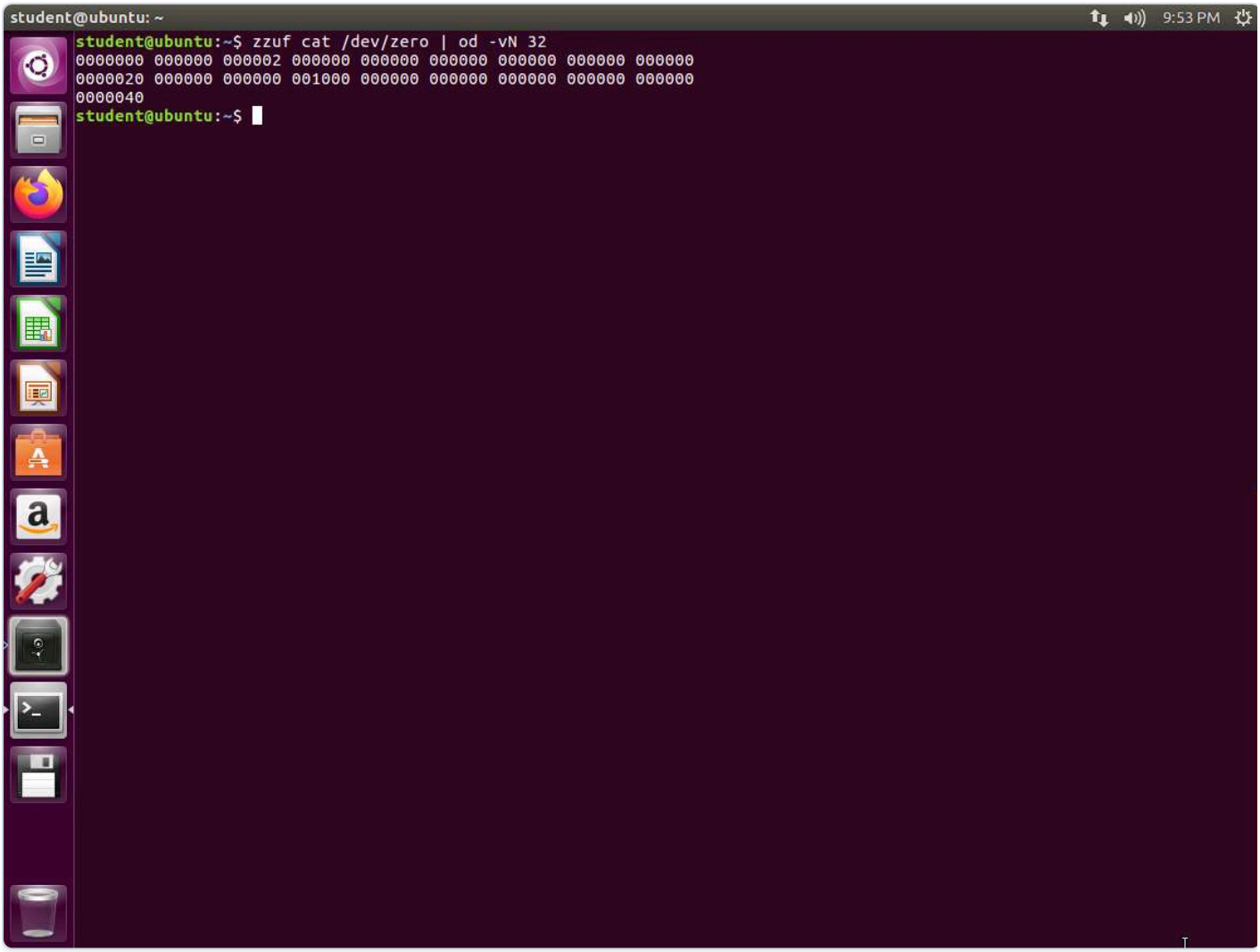
1. View raw zero-byte stream output using hex dump:
   hd -vn 32 /dev/zero



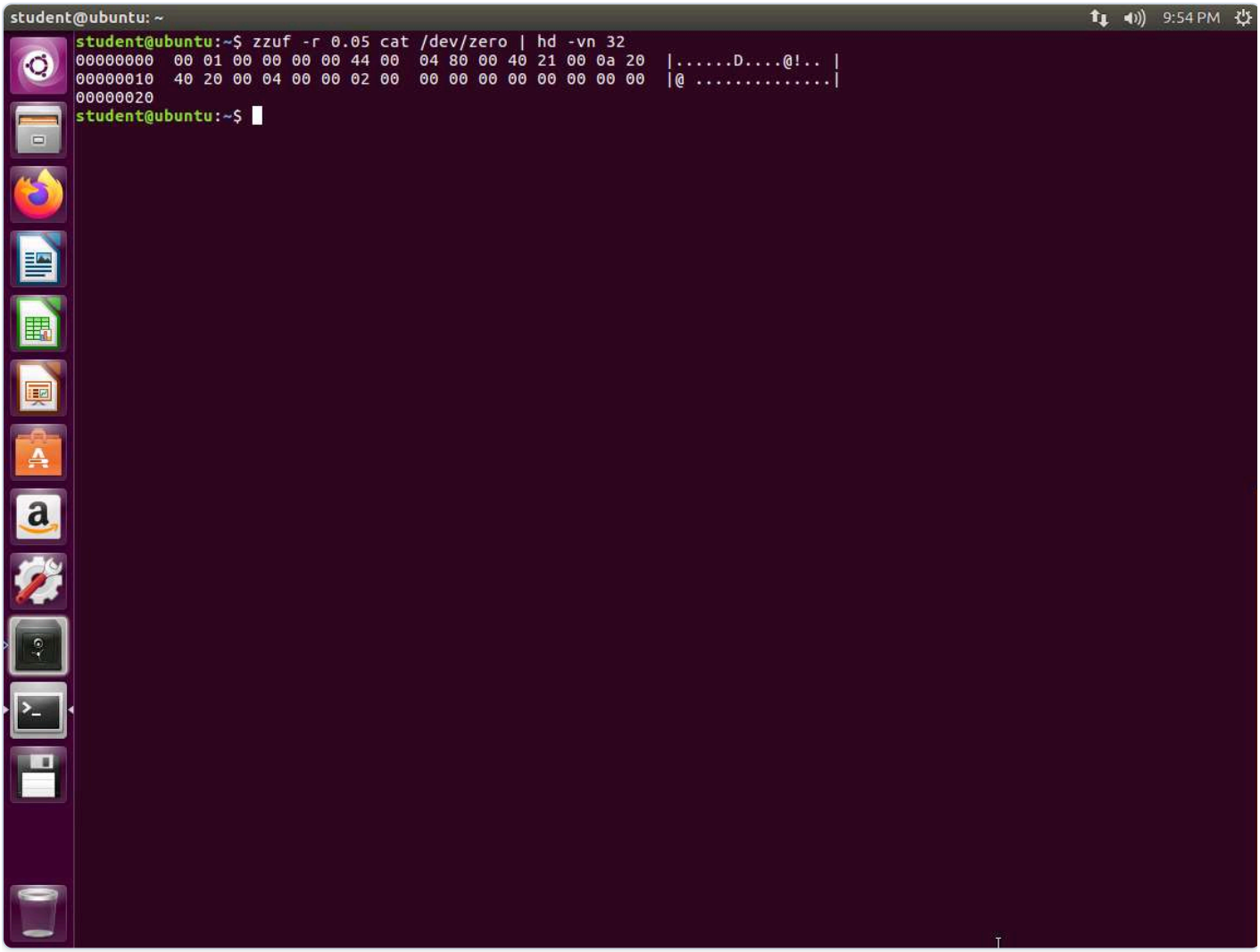
1. Fuzz `cat /dev/zero` stream through `zzuf` pipe:
   zzuf cat /dev/zero | hd -vn 32



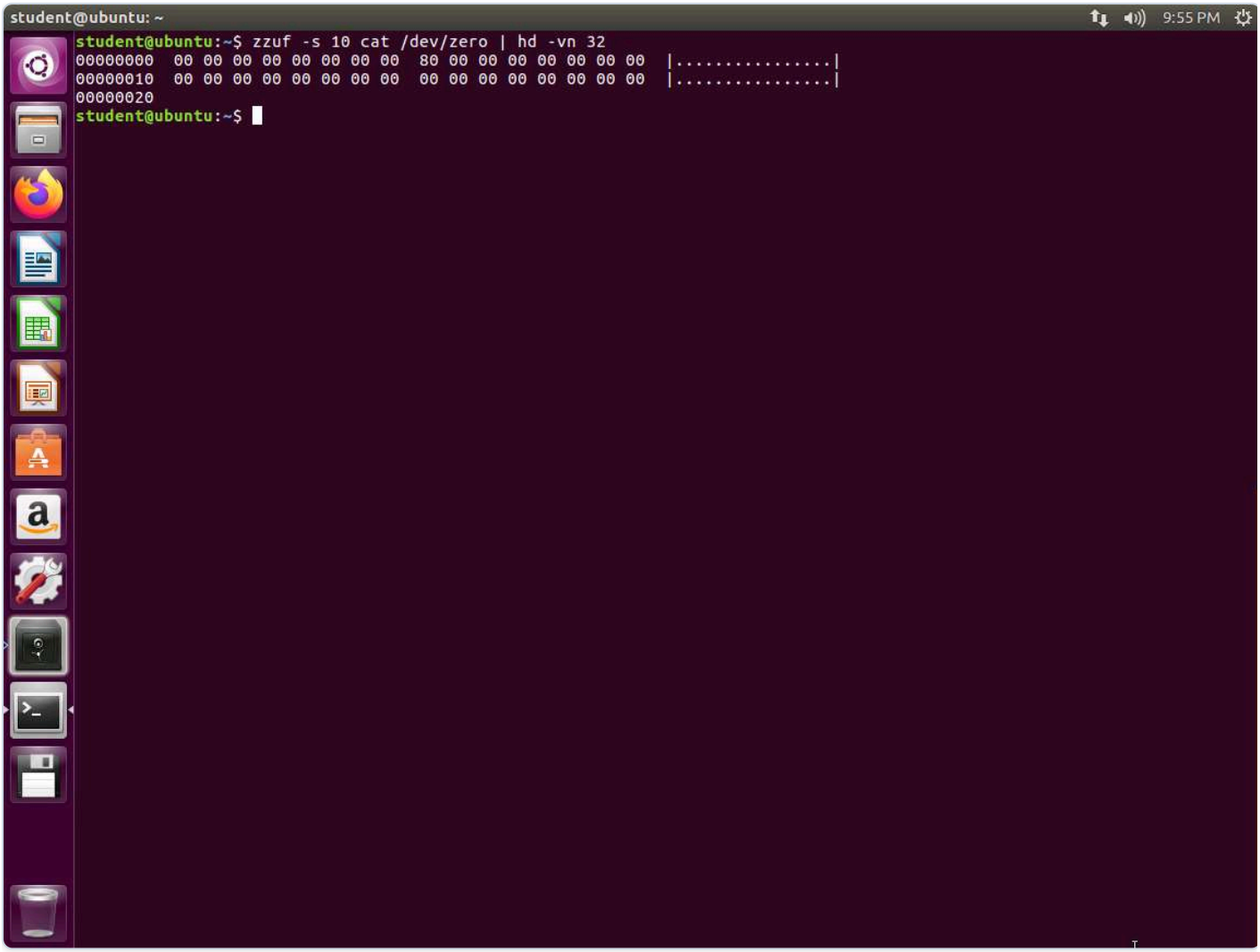
1. Test octal format output verifying reproducible mutation behavior:
   zzuf cat /dev/zero | od -vN 32



1. Increase mutation ratio to 5% (`-r 0.05`):
   zzuf -r 0.05 cat /dev/zero | hd -vn 32



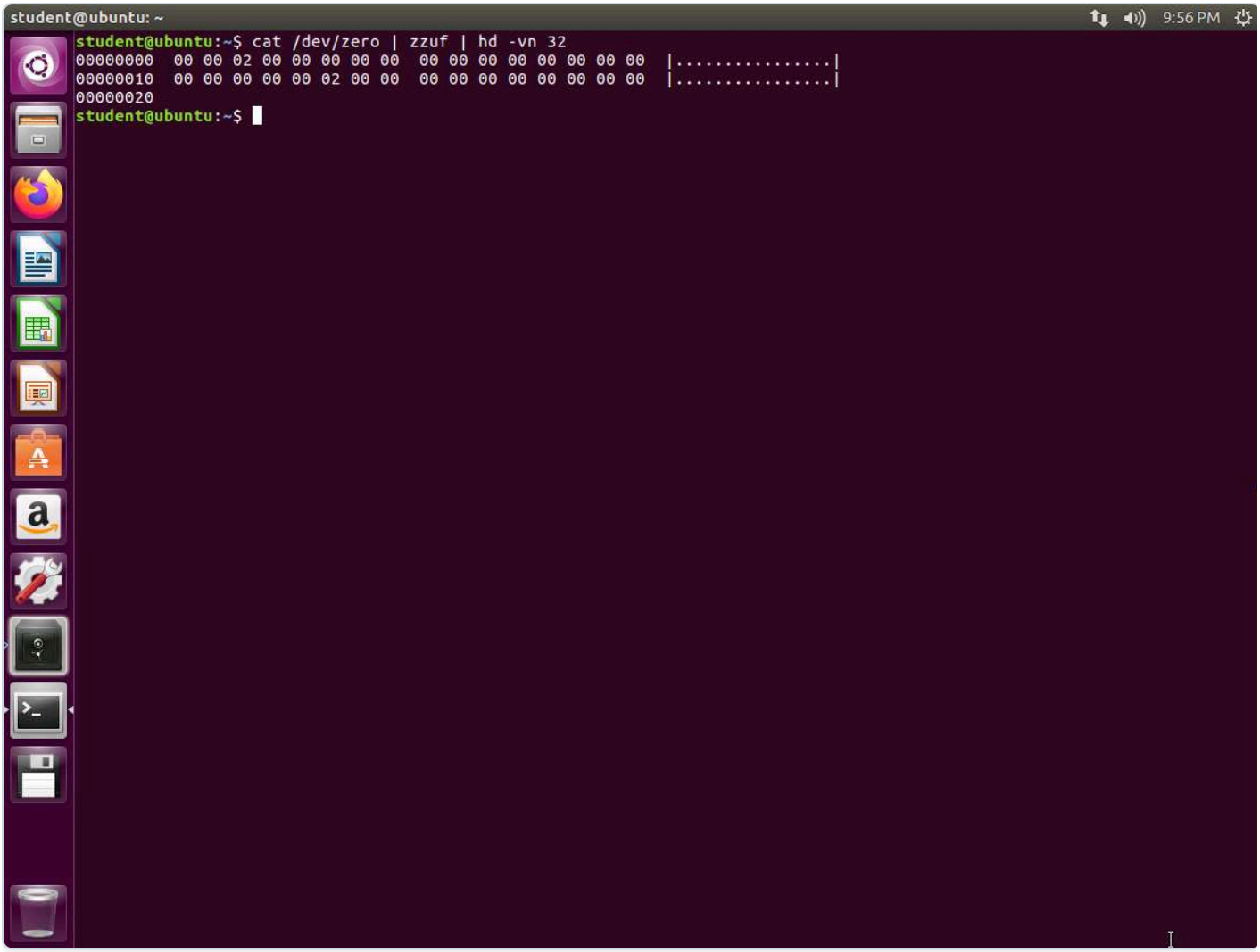
1. Change random generator seed to 10 (`-s 10`):
   zzuf -s 10 cat /dev/zero | hd -vn 32



1. Fuzz pipe streams directly and output to file:

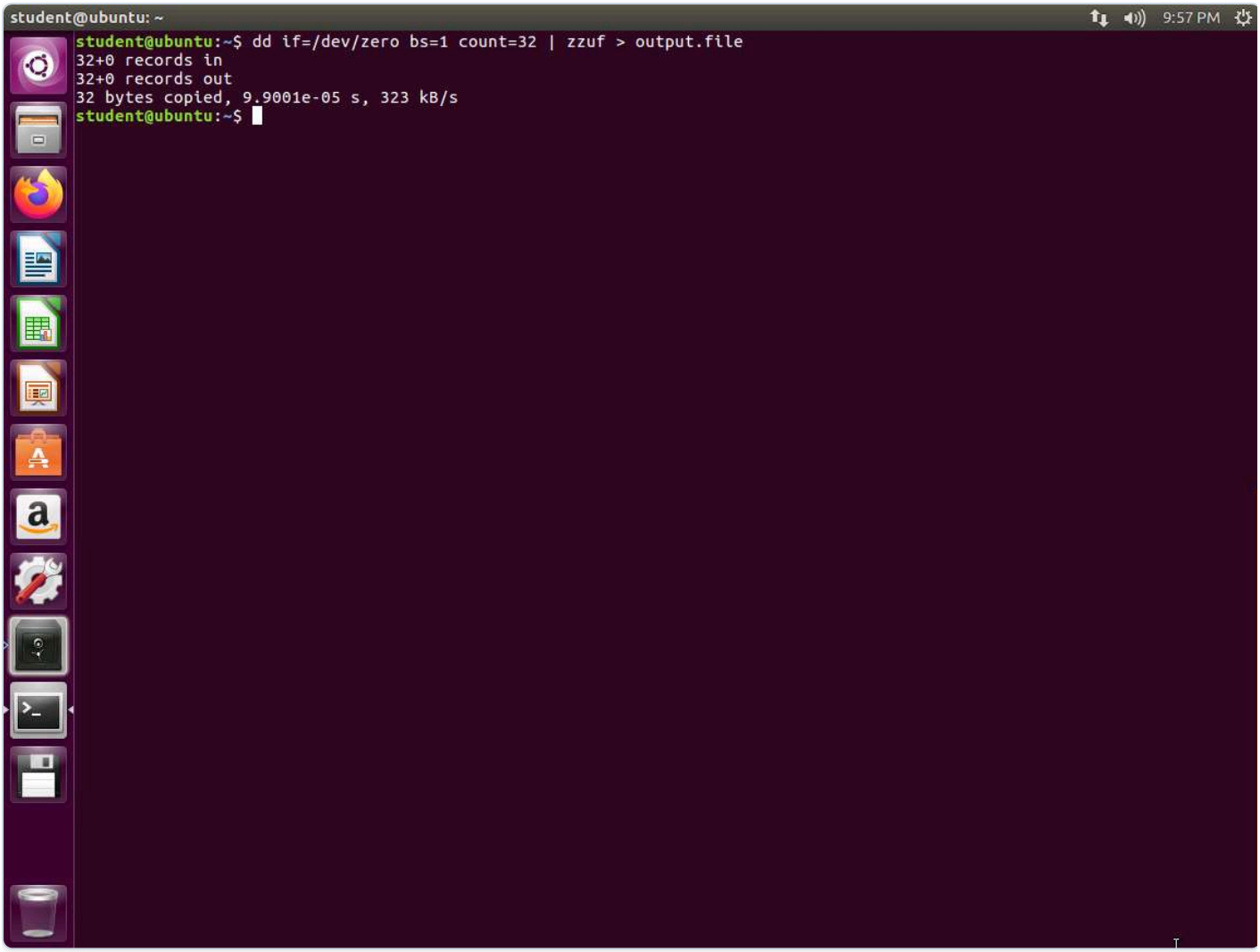
   cat /dev/zero | zzuf | hd -vn 32

   dd if=/dev/zero bs=1 count=32 | zzuf > output.file



1. Inspect fuzzed file output in hex:

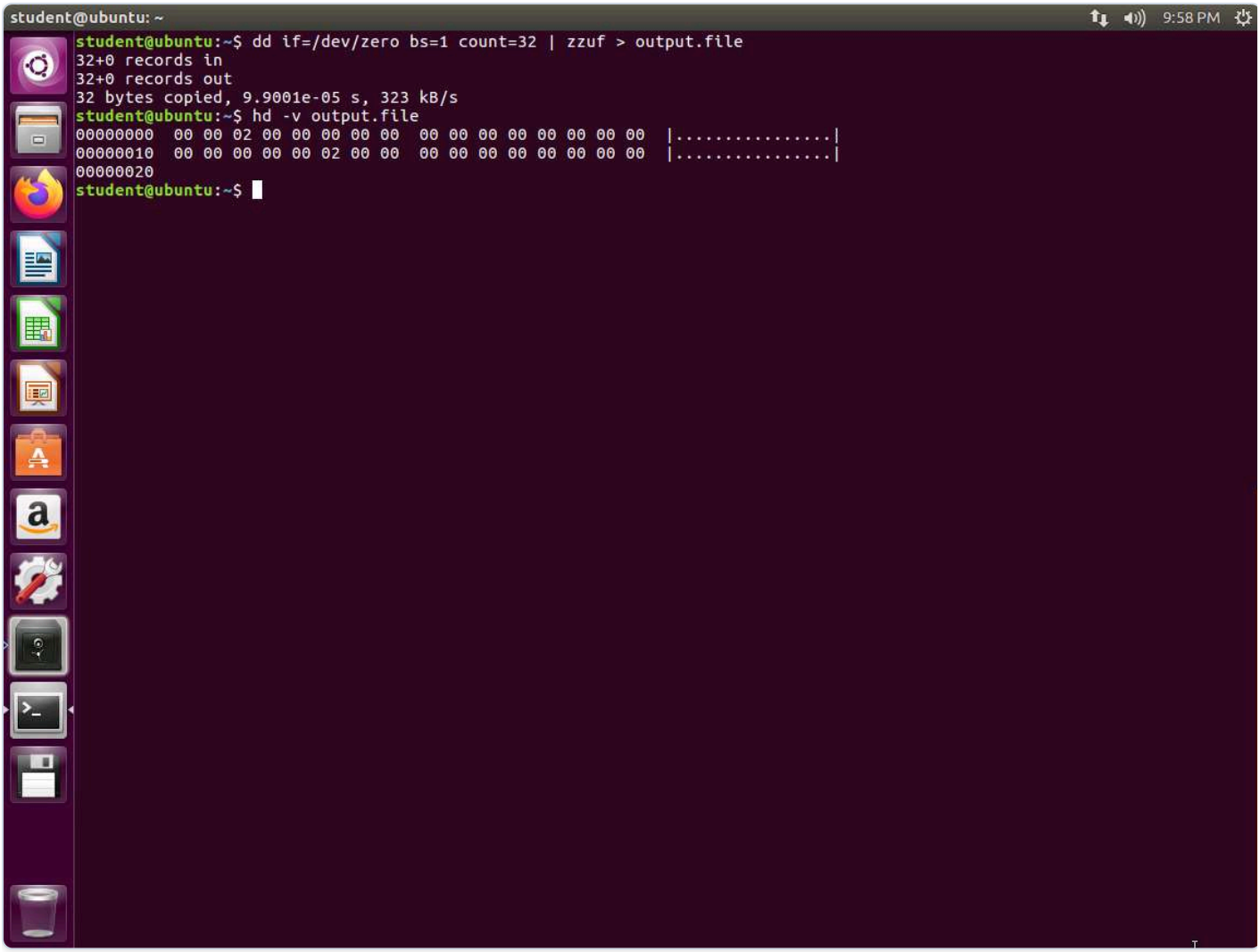
   hd -v output.file

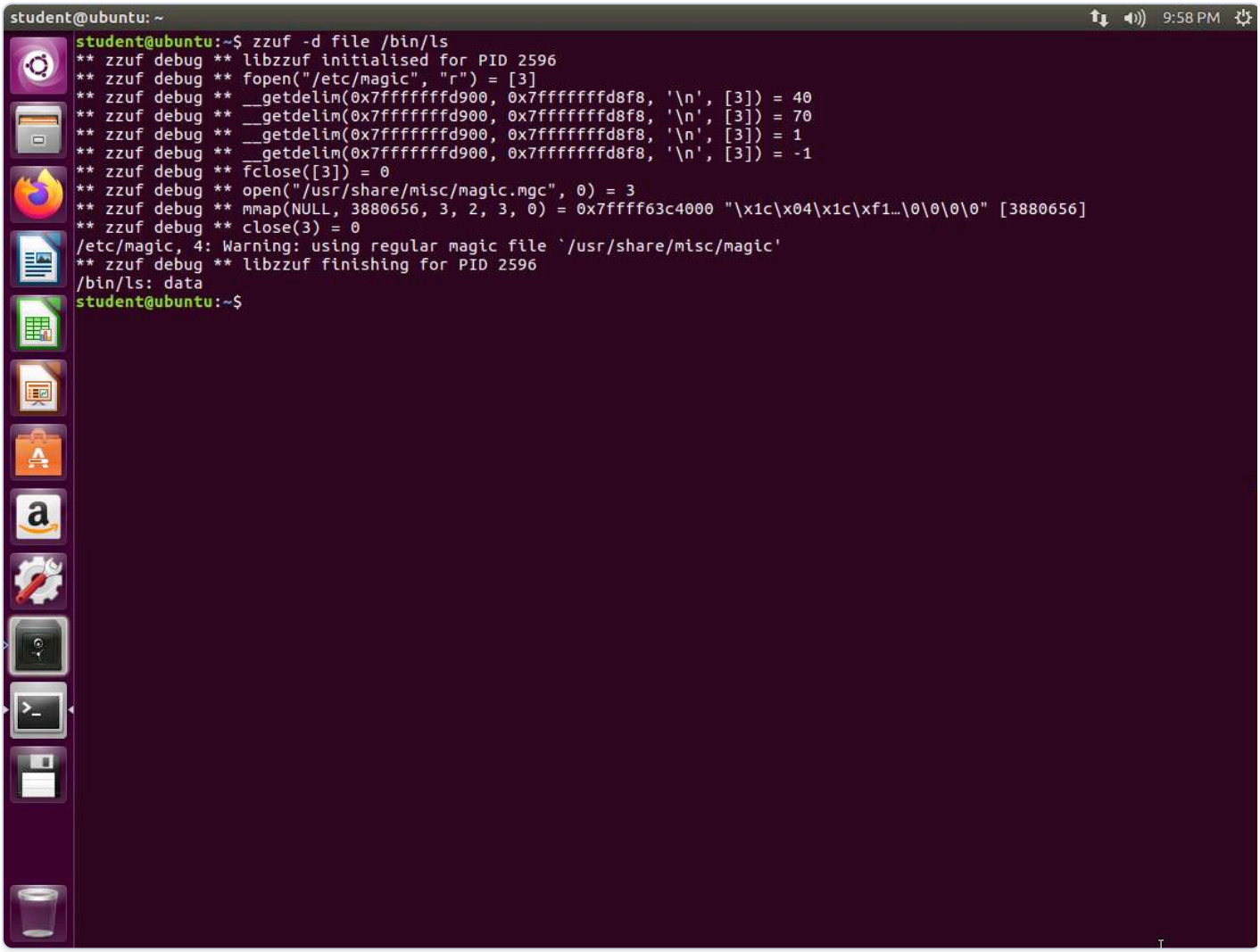


1. Test debug mode and system path exclusions:

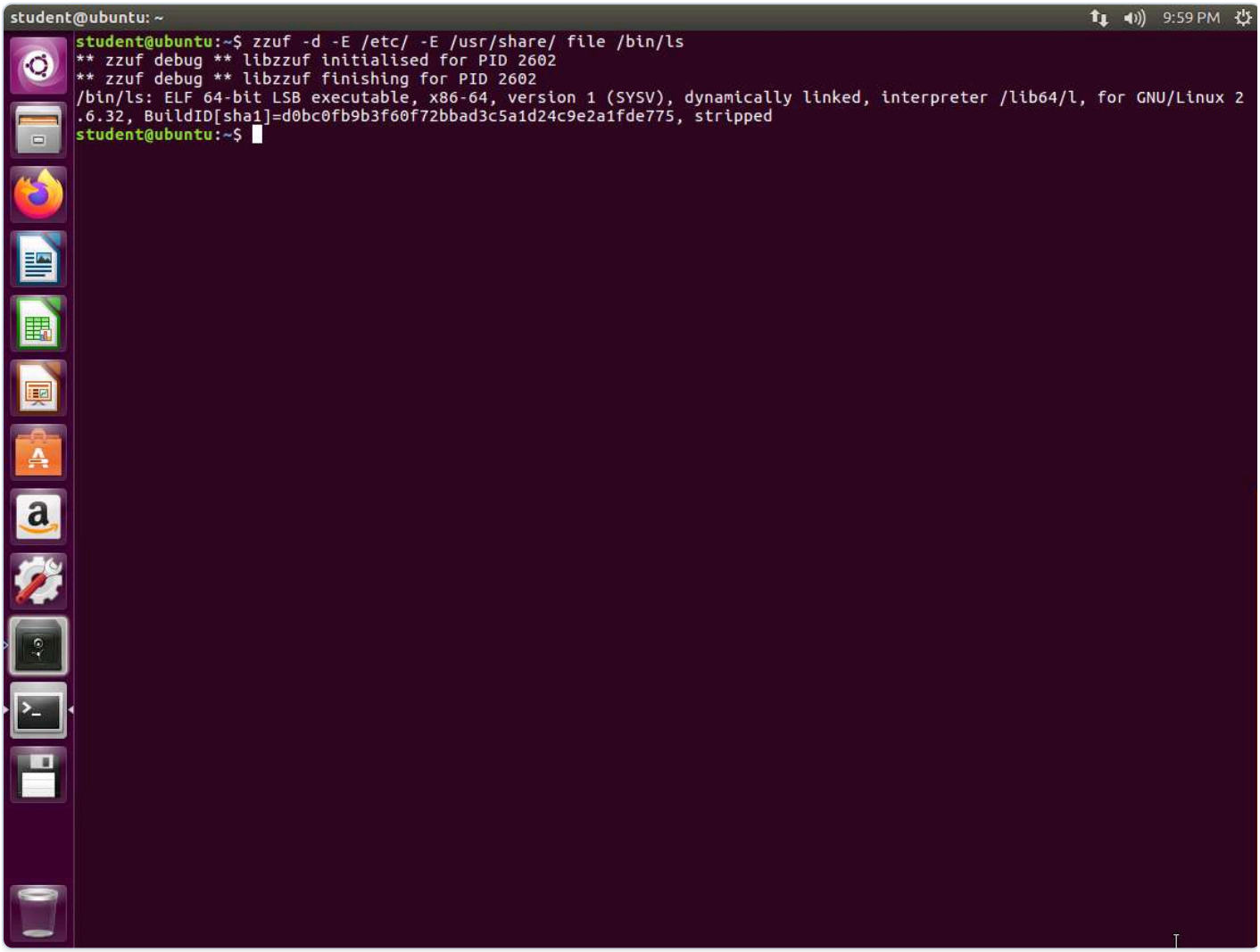
   zzuf -d file /bin/ls

   zzuf -d -E /etc/ -E /usr/share/ file /bin/ls


student@ubuntu: ~
9:58 PM
student@ubuntu:~$ dd if=/dev/zero bs=1 count=32 | zzuf > output.file
32+0 records in
32+0 records out
32 bytes copied, 9.9001e-05 s, 323 kB/s
student@ubuntu:~$ hd -v output.file
00000000 00 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 |................|
00000010 00 00 00 00 00 02 00 00 00 00 00 00 00 00 00 00 |................|
00000020
student@ubuntu:~$



1. Test command line file inclusion flag (`-c`):
   zzuf -d -c file /bin/ls

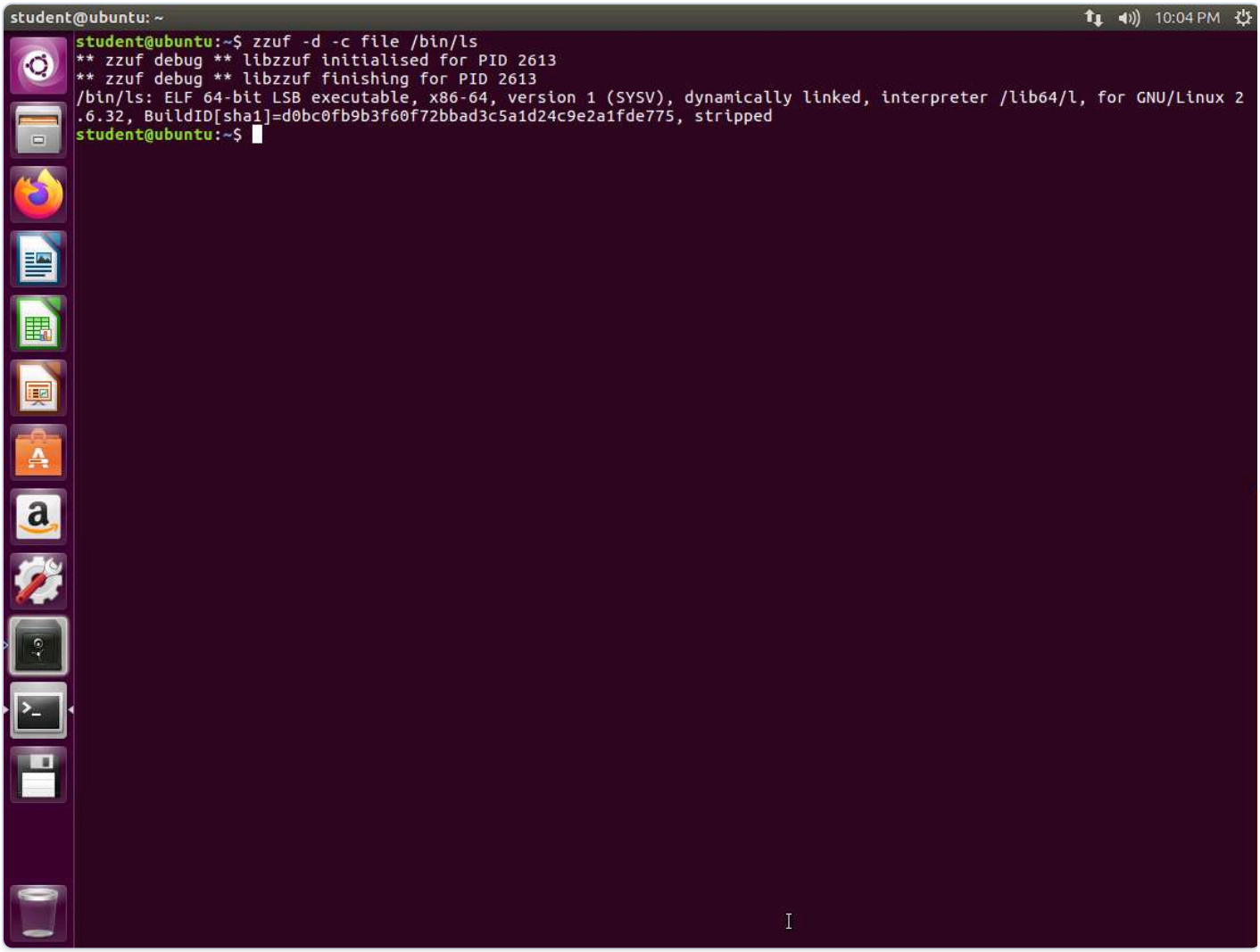


1. Navigate to sample code folder and generate batch variants:

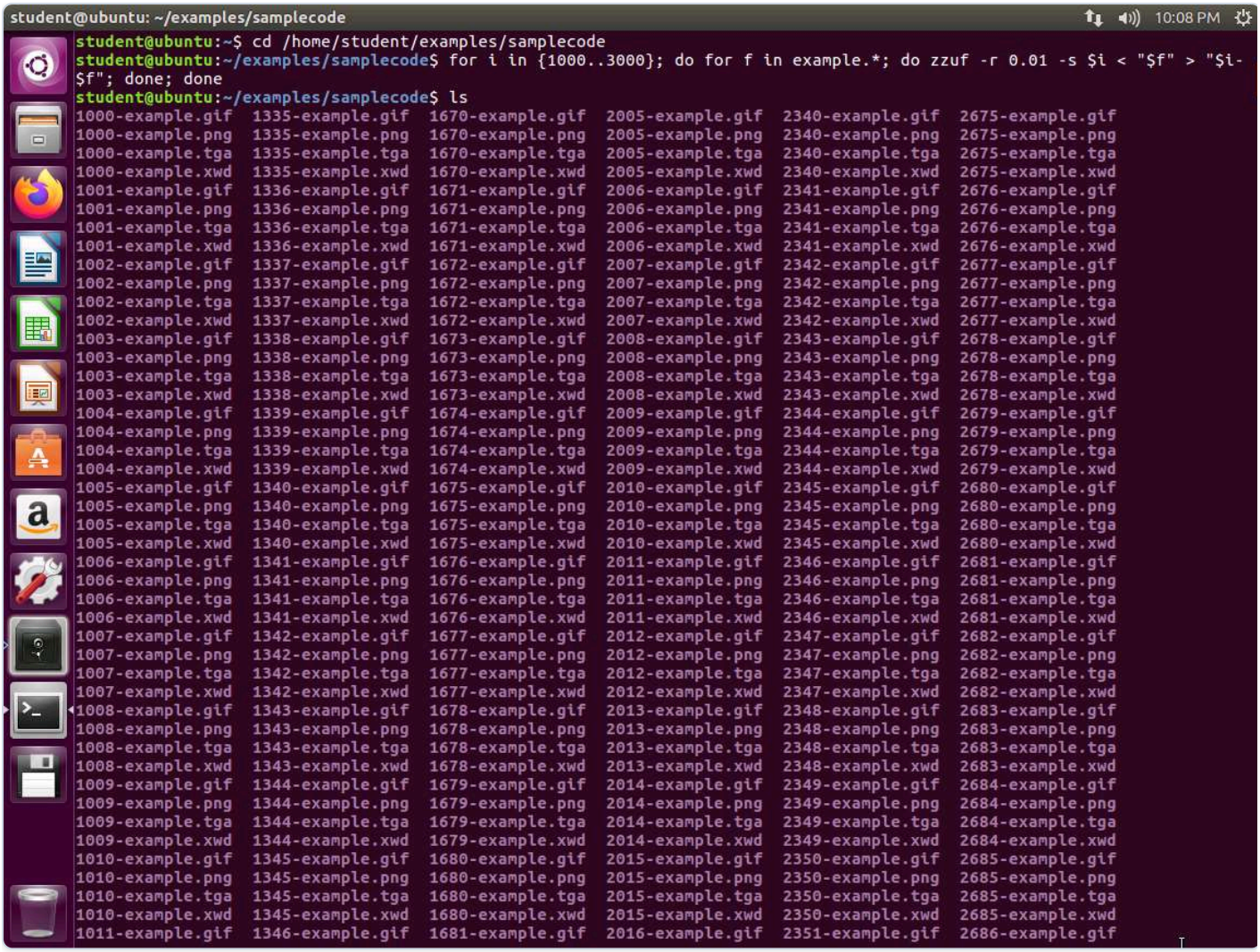
   cd /home/student/examples/samplecode

   for i in {1000..3000}; do for f in example.*; do zzuf -r 0.01 -s $i < "$f" > "$i-$f"; done; done

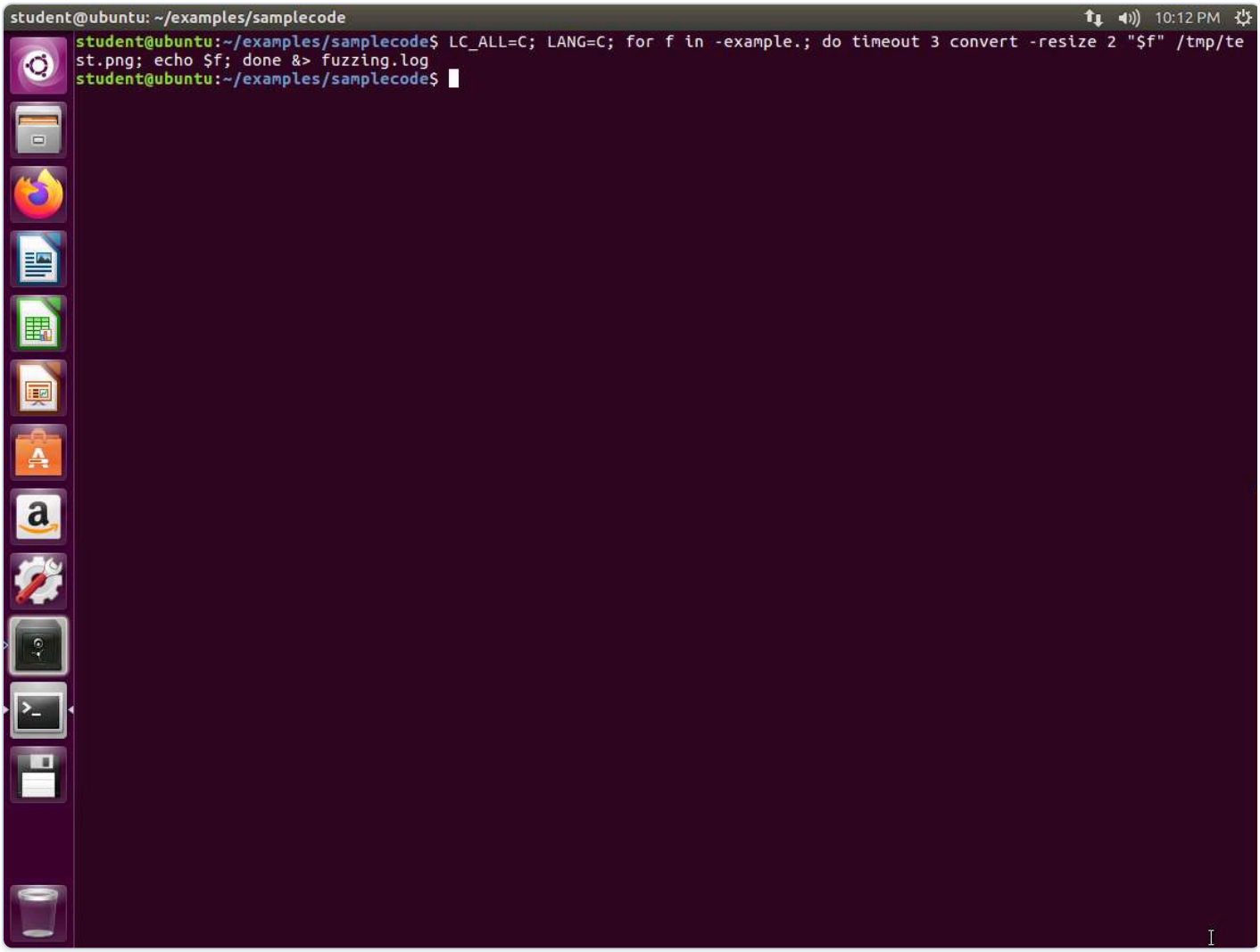
   ls



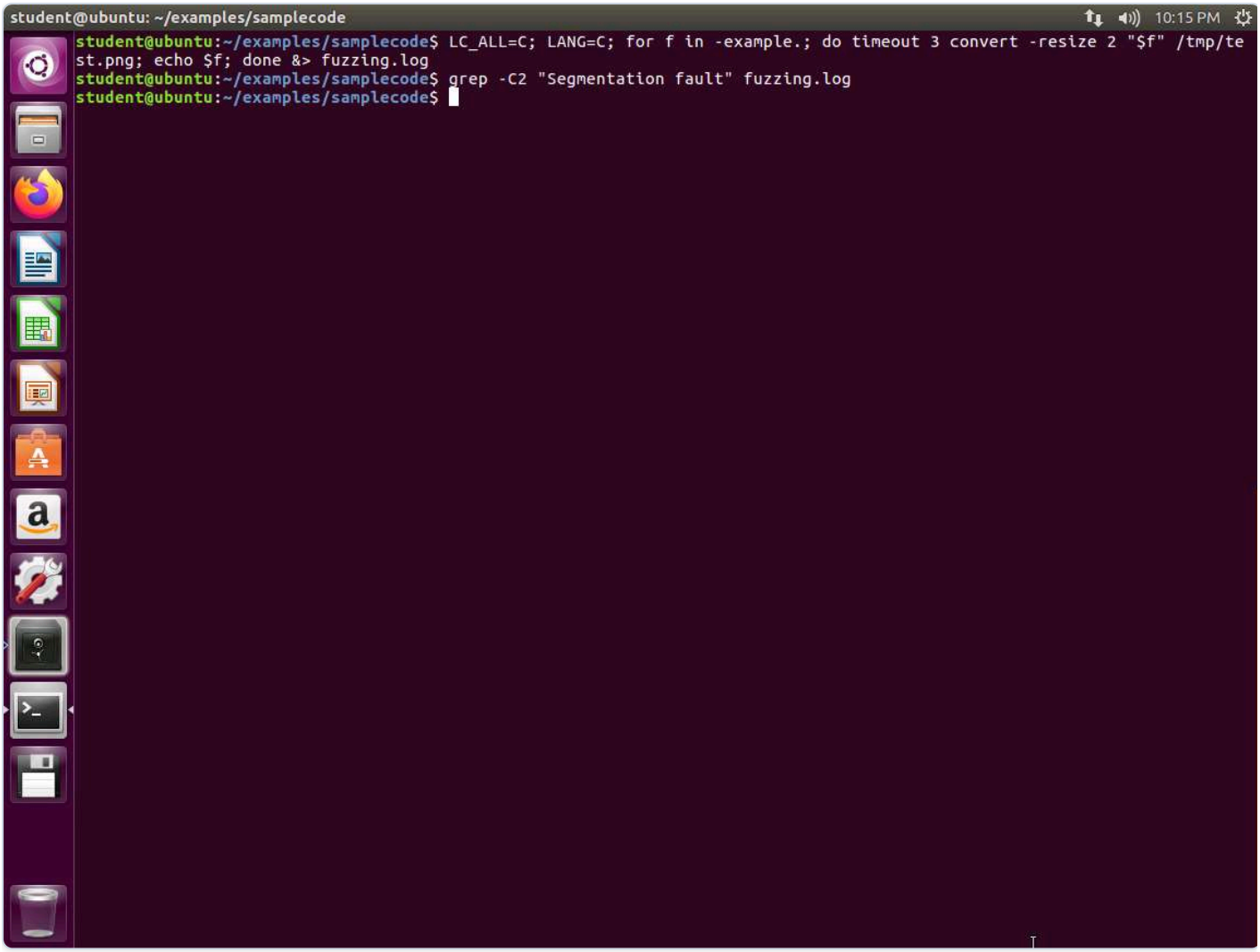
1. Batch process malformed files with ImageMagick convert logging errors:
   LC_ALL=C; LANG=C; for f in *-example.*; do timeout 3 convert -resize 2 "$f" /tmp/test.png; echo $f; done &> fuzzing.log



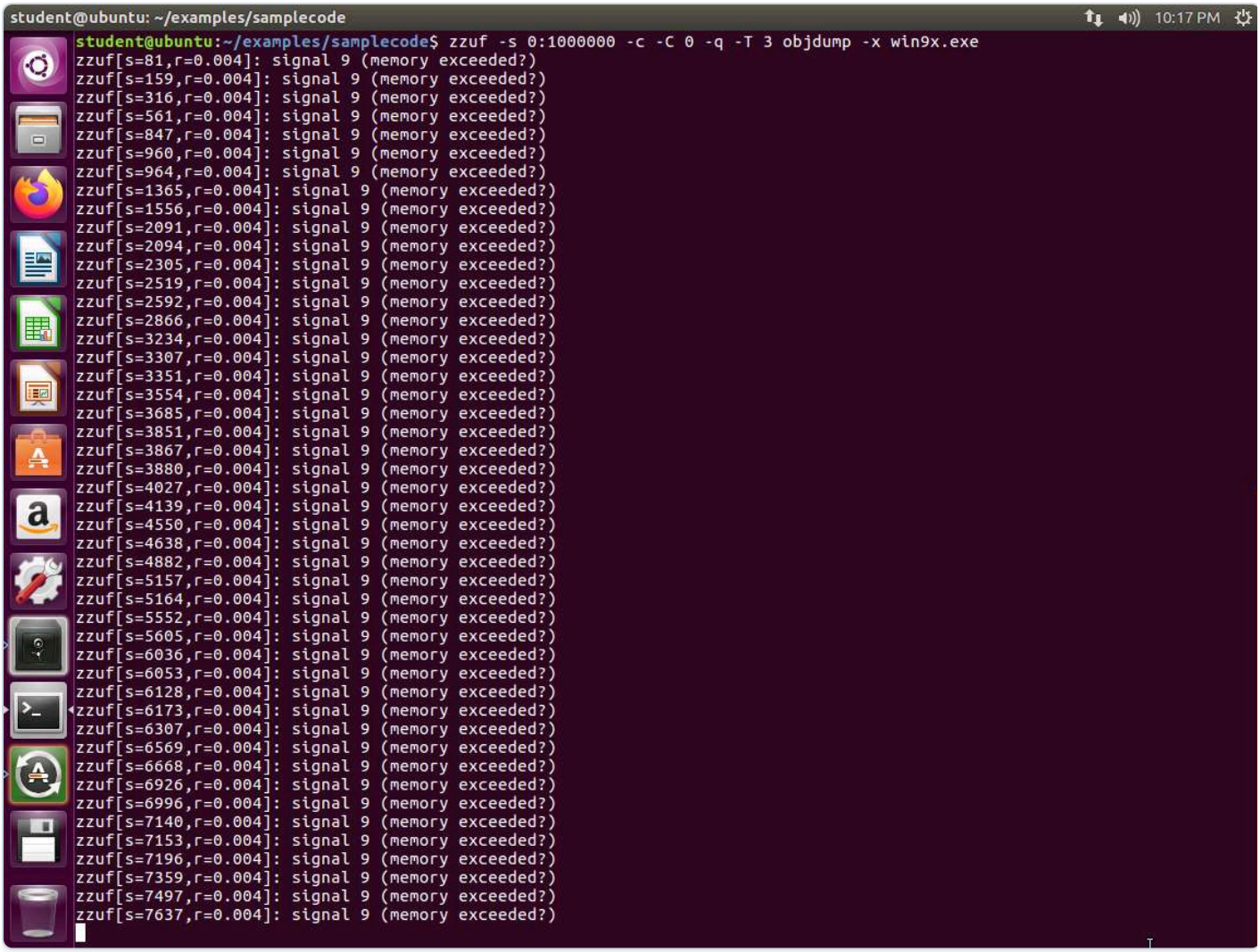
1. Search log file for segmentation fault crashes:
   grep -C2 "Segmentation fault" fuzzing.log

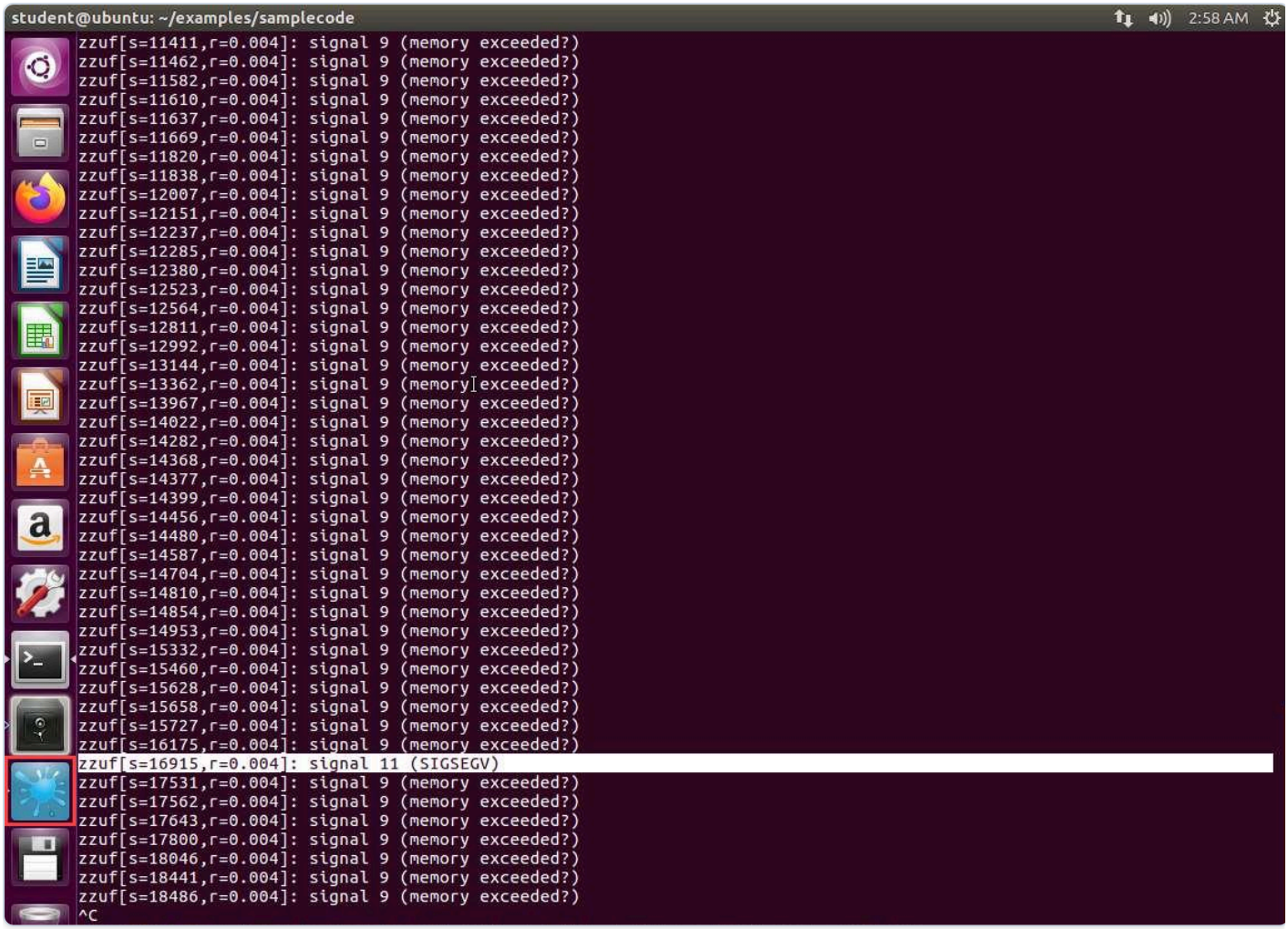


1. Execute 1,000,000 seed fuzzing loop against `objdump` inspecting **win9x.exe**:
   zzuf -s 0:1000000 -c -C 0 -q -T 3 objdump -x win9x.exe



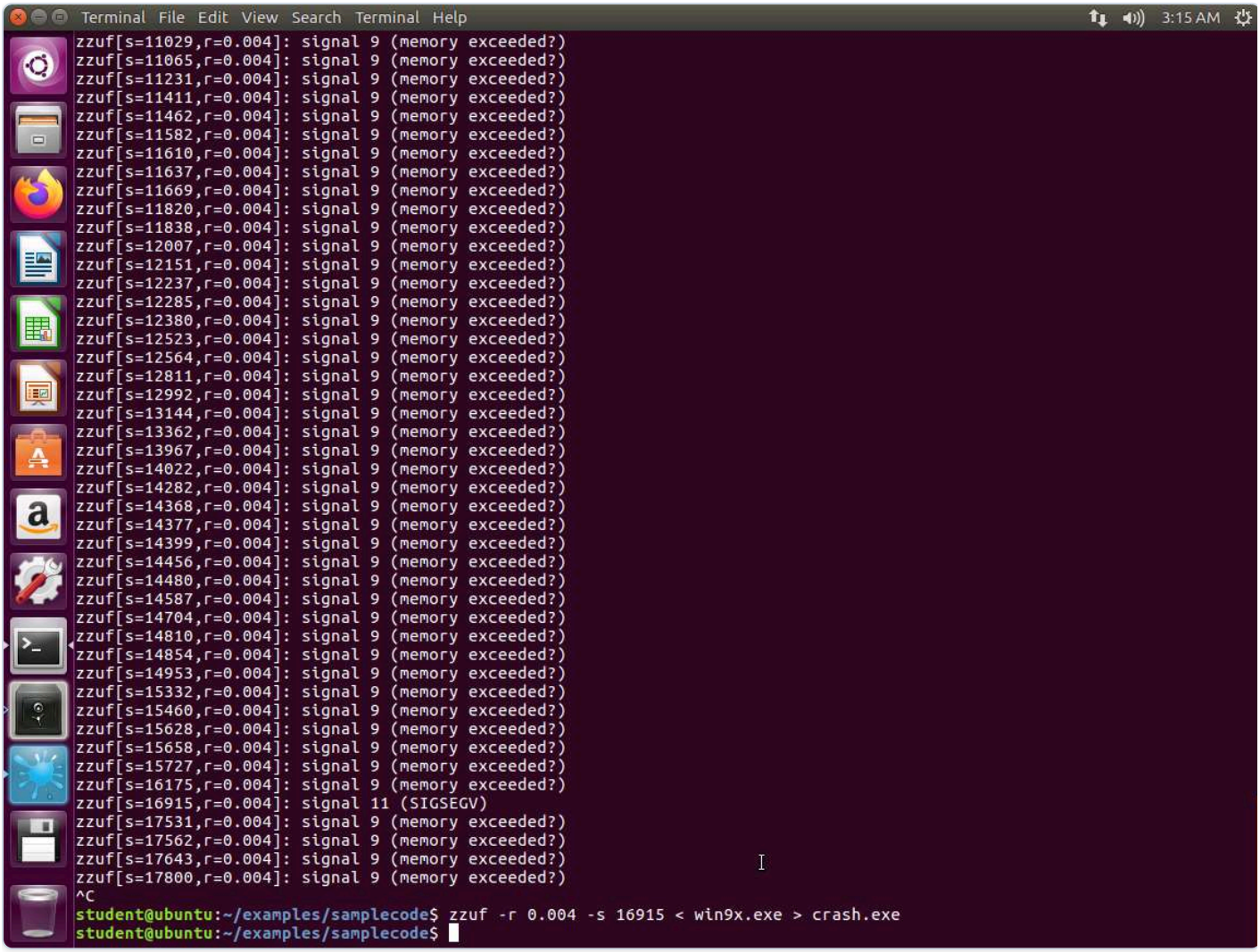
1. Observe crash alert pop-up (signal 11 / SIGSEGV).


student@ubuntu: ~/examples/samplecode
10:17 PM
student@ubuntu:~/examples/samplecode$ zzuf -s 0:1000000 -c -C 0 -q -T 3 objdump -x win9x.exe
zzuf[s=81,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=159,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=316,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=561,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=847,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=960,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=964,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=1365,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=1556,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2091,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2094,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2305,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2519,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2592,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=2866,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3234,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3307,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3351,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3554,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3685,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3851,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3867,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=3880,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=4027,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=4139,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=4550,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=4638,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=4882,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=5157,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=5164,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=5552,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=5605,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6036,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6053,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6128,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6173,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6307,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6569,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6668,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6926,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=6996,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7140,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7153,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7196,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7359,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7497,r=0.004]: signal 9 (memory exceeded?)
zzuf[s=7637,r=0.004]: signal 9 (memory exceeded?)

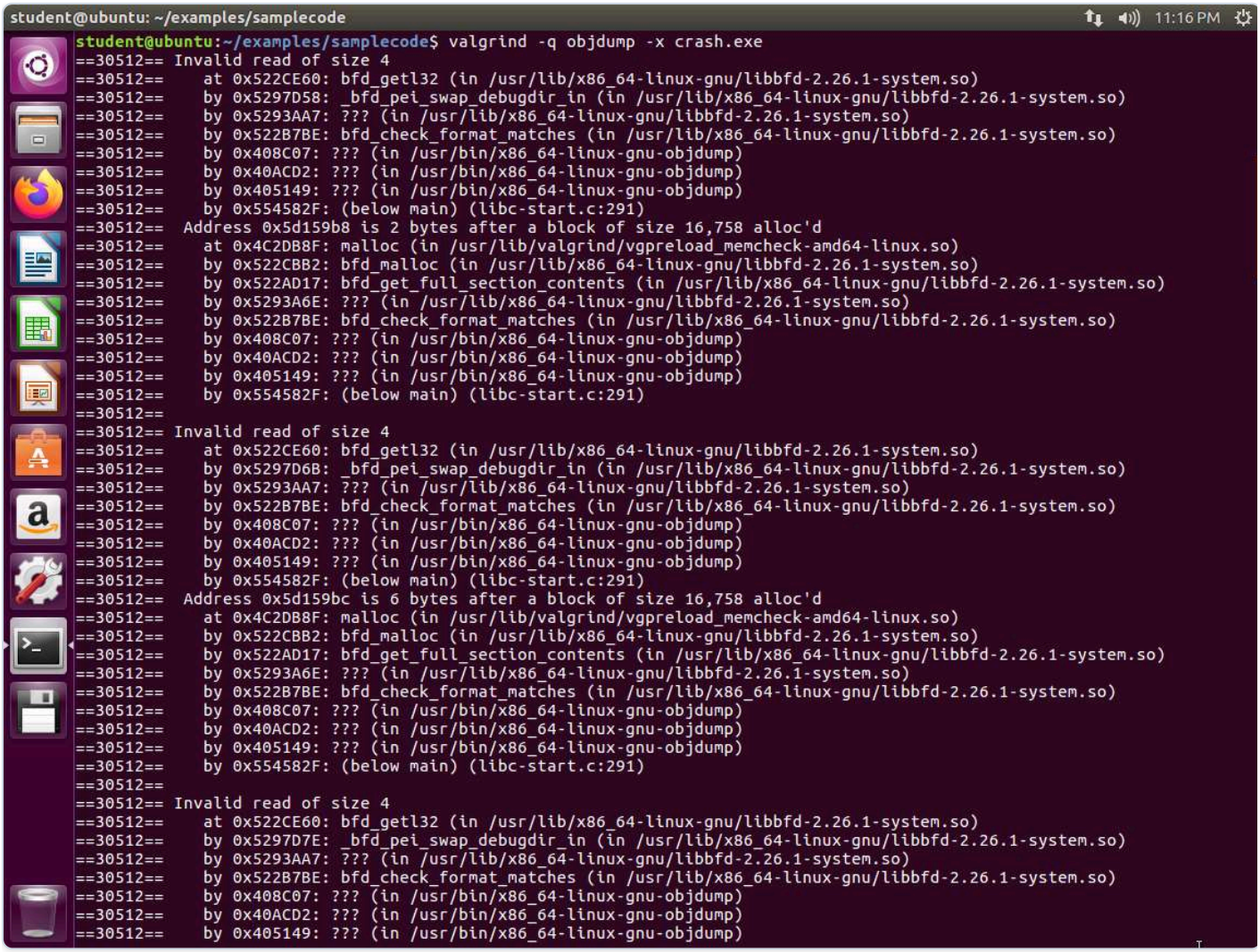


1. Extract payload using crashing seed (`s=16915`):

   zzuf -r 0.004 -s 16915 < win9x.exe > crash.exe



1. Perform root cause memory crash analysis with Valgrind:
   valgrind -q objdump -x crash.exe



This concludes the lab exercise.

> **IMPORTANT:** ❓ **Question 11.12.1.1** In Ub20 Fuzzing machine, perform fuzzing using the zzuf tool available at /home/student/Downloads/fuzzing/zzuf. Enter the signal number that indicates the segmentation fault. Enter only the signal number. [!SUCCESS] ✅ **Answer:** **9**